



VINUNIVERSITY

Lab 07: Search trees, balanced trees, and algorithm for tree traversal.

Week 07, Apr 13 2026

**College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring**

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should **upload your .java file(s) to Codeforces before the end of the laboratory session** unless the instructor gave a specified deadline. **No Canvas or Codeforces submission** will get 0 point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question.
- Late submission of lab practice without an approved extension will incur the following penalties:
 1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
 2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
 3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.
- **Navigate to the Lab:** Open the specific link provided for this week's lab: <https://vinuni26oop.contest.codeforces.com/>.

Problem 1: BST Structure

The orientation team is preparing a campus map index. Each important location is stored as a numeric location ID inside a BST, and they need a quick structural report before publishing the map.

Write a Java program from scratch that reads the input values, builds one binary search tree (BST), and then analyzes the final tree structure.

Detailed Logic: Implement the algorithm step by step:

- Create a node structure to store one location ID in the BST.
- Read all input values and insert them one by one in the given order.
- After the BST is fully built, determine which key is at the root.
- Count how many nodes in the final tree are leaf nodes.
- Compute the height of the final tree, which is the number of edges on the longest path from the root to a leaf. A tree with one node has height 0.
- Perform an inorder traversal and prepare the output string.

Input

- First line: integer N
- Second line: N integers representing location IDs in insertion order

Output

Print exactly four lines in this order:

- Line 1: the root key
- Line 2: the number of leaf nodes
- Line 3: the height of the tree
- Line 4: the inorder traversal, with keys separated by one space

Do not print any extra words or labels.

Examples

Test Case 1:

Input:

7

50 30 70 20 40 60 80

Output:

50

4

```
2
20 30 40 50 60 70 80
```

Test Case 2:

Input:

```
1
42
```

Output:

```
42
1
0
42
```

Problem 2: BST Operations with Commands

A separate navigation service already has a BST of location IDs. During testing, the service receives live commands to add newly opened locations and search for requested location IDs.

This problem is an extension of Problem 1. Reuse the same BST foundation (node structure + insertion behavior), then add command processing so one BST can be updated and queried over time.

Write a Java program that keeps one BST up to date while processing insert and search commands one by one.

Implementation note:

- Reuse your BST foundation from Problem 1.
- Process commands online on one persistent BST (insert/search).
- Print command results and the final inorder traversal exactly as specified.

Input

- First line: integers N M
- Second line: N integers already present in the BST
- Next M lines: one command per line
 - 1 key — insert key
 - 2 key — search for key

Output

For each command:

- If the command is 1 key , print 1 if the insertion succeeds, otherwise print 0.
- If the command is 2 key , print 1 if the key exists, otherwise print 0.

After processing all commands, print one final line containing the inorder traversal, with keys separated by one space. Do not print any extra words or labels.

Examples

Test Case 1:

Input:

3 4

40 20 60

1 50

2 50

1 20

2 100

Output:

1

1

0

0

20 40 50 60

Test Case 2:

Input:

5 5

20 10 30 5 15

2 25

2 18

1 25

1 10

2 5

Output:

0

0

1

0

1

5 10 15 20 25 30

Problem 3: BST Traversal

Another campus subsystem needs traversal exports from a BST of location IDs so route planning, auditing, and reporting tools can all reuse the same tree.

Write a Java program from scratch that builds a BST from the input and then prints the preorder, inorder, and postorder traversals of that tree.

Implementation note:

- Build one BST from the given insertion order.

- Infer the required traversal helpers from the output specification.
- Print preorder, inorder, and postorder exactly as required.

Input

- First line: integer N
- Second line: N integers representing location IDs in insertion order

Output

Print exactly three lines in this order:

- Line 1: the preorder traversal, with keys separated by one space
- Line 2: the inorder traversal, with keys separated by one space
- Line 3: the postorder traversal, with keys separated by one space

Do not print any extra words or labels.

Examples

Test Case 1:

Input:

7

40 20 60 10 30 50 70

Output:

40 20 10 30 60 50 70

10 20 30 40 50 60 70

10 30 20 50 70 60 40

Test Case 2:

Input:

4

25 10 40 30

Output:

25 10 40 30

10 25 30 40

10 30 40 25

Problem 4: BST Balance

As VinUni Campus Navigators, your BST index stores important campus landmarks by their location IDs. To ensure that route calculations stay fast even as the campus grows, the final BST must be height-balanced (AVL-style: for every node, the heights of the left and right subtrees differ by at most 1).

Write a Java program from scratch that reads the input values, builds one binary search tree (BST), and then checks if the resulting tree is height-balanced.

Implementation note:

- Build one BST from the given insertion order.
- Determine balance bottom-up: for each node, compare the heights of its left and right subtrees; if any difference is greater than 1, the BST is not balanced.
- Print exactly one line: `BALANCED` or `NOT BALANCED`.

Input

- First line: integer `N`
- Second line: `N` integers representing location IDs in insertion order

Output

Print exactly one line: - `BALANCED` if the BST is height-balanced - `NOT BALANCED` otherwise

Examples

Test Case 1:

Input:

3

3 2 1

Output:

`NOT BALANCED`

Test Case 2:

Input:

4

3 2 1 4

Output:

`BALANCED`